GRAPH DATA STRUCTURES & ALGORITHMS

Complete Java DSA Master Notes | High-Tech Interview Preparation

Scope: Graph fundamentals to advanced interview patterns, with Java implementations, complexity analysis, dry-run guidance, and common traps.

Designed as a comprehensive graph syllabus for B.Tech CSE / software engineering interviews. Java 17+ syntax is used throughout. This is a graph-only guide; unrelated DSA topics are excluded.

1. Graph Fundamentals

Graph: A graph G = (V, E) consists of a set of vertices (nodes) V and edges E connecting pairs of vertices.

• Undirected graph: edge {u,v} has no direction. Directed graph (digraph): edge u → v has a direction.

• Weighted graph: every edge has a cost/weight. Unweighted graph: all edges are treated equally (often weight 1).

• Simple graph: no self-loops and no parallel edges. Multigraph: parallel/multiple edges may exist. Pseudograph may also allow self-loops.

• Self-loop: edge from a vertex to itself. Parallel edges: two or more edges joining the same pair of vertices.

• Adjacent vertices share an edge. Incident edge touches a vertex. Neighbours of u are vertices connected to u.

• Degree in an undirected graph: number of incident edges; a self-loop contributes 2. In-degree: incoming directed edges. Out-degree: outgoing directed edges.

• Isolated vertex has degree 0. Pendant/leaf vertex has degree 1. Null graph has vertices but no edges. Trivial graph has exactly one vertex and no edges.

• Complete graph K_n: every distinct pair of vertices is connected; edges = n(n–1)/2. Path graph P_n has n–1 edges. Cycle graph C_n has n edges (n≥3).

• Connected graph: every pair of vertices has a path between them. A disconnected graph has two or more connected components.

• Walk may repeat vertices/edges; trail does not repeat edges; path does not repeat vertices; cycle is a closed path (terminology may vary by textbook).

• Tree: connected, undirected, acyclic graph. A tree with V vertices has exactly V–1 edges; there is exactly one simple path between any two vertices.

Handshaking Lemma: For an undirected graph, Σ degree(v) = 2E. Therefore, the number of odd-degree vertices is always even.

Directed degree identity: Σ indegree(v) = Σ outdegree(v) = E.

2. Graph Classification & Special Graphs

• Bipartite graph: vertices can be split into two sets so every edge joins opposite sets. A graph is bipartite iff it has no odd-length cycle.

• Complete bipartite graph K(m,n): every vertex in the first set connects to every vertex in the second; edges = mn.

• DAG: directed acyclic graph. It has at least one topological ordering.

• Regular graph: every vertex has the same degree. k-regular means each vertex has degree k.

• Subgraph: formed from subsets of vertices and edges of another graph. Induced subgraph includes all original edges between selected vertices.

• Complement graph: connects exactly those distinct vertex pairs that were not adjacent in the original simple graph.

• Planar graph can be drawn without edge crossings. For a connected planar graph: V – E + F = 2 (Euler's formula).

3. Graph Representations in Java

A. Adjacency Matrix

```
int n = 5;
int[][] adj = new int[n][n];
adj[0][1] = 1;
adj[1][0] = 1; // undirected
// Weighted graph: store weight instead of 1.
```

Space O(V²). Checking whether an edge exists is O(1). Iterating neighbours of a vertex is O(V). Best for dense graphs or small fixed V.

B. Adjacency List (most common in interviews)

```
import java.util.*;

int n = 5;
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
```

```
graph.get(0).add(1);
graph.get(1).add(0); // undirected edge
```

Space O(V+E). Iterating neighbours is O(degree(u)). Best for sparse graphs. For weighted graphs, store Edge(to, weight) objects.

C. Edge List

```
int[][] edges = {
    {0, 1, 4}, {1, 2, 7}, {0, 2, 9}
}; // {from, to, weight}
```

Space O(E); convenient for Kruskal and Bellman–Ford. Matrix vs list vs edge-list is a common interview question.

4. Build Reusable Java Graph Structures

```
import java.util.*;

class Graph {
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    List<List<Edge>> adj;
    Graph(int n) {
        adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    }
    void addUndirectedEdge(int u, int v, int w) {
        adj.get(u).add(new Edge(v, w));
        adj.get(v).add(new Edge(u, w));
    }
    void addDirectedEdge(int u, int v, int w) {
        adj.get(u).add(new Edge(v, w));
    }
}
```

Use vertices 0 to n–1 unless the problem specifies otherwise. For disconnected graphs, start traversal from every unvisited vertex.

5. BFS — Breadth-First Search

Idea: Explore level by level using a FIFO queue. In an unweighted graph, BFS finds the minimum number of edges from a source to every reachable vertex.

```
static void bfs(List<List<Integer>> g, int src) {
    int n = g.size();
    boolean[] vis = new boolean[n];
    Queue<Integer> q = new ArrayDeque<>();
    vis[src] = true;
    q.offer(src);

    while (!q.isEmpty()) {
        int u = q.poll();
        System.out.print(u + " ");
        for (int v : g.get(u)) {
            if (!vis[v]) {
                vis[v] = true; // mark when enqueued
                q.offer(v);
            }
        }
    }
}
```

Complexity: O(V+E) time, O(V) extra space. Mark visited when enqueuing to avoid duplicate queue entries.

BFS shortest distance in an unweighted graph

```
static int[] bfsDistance(List<List<Integer>> g, int src) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, -1);
    Queue<Integer> q = new ArrayDeque<>();
    dist[src] = 0;
    q.offer(src);

    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : g.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.offer(v);
            }
        }
    }
    return dist;
}
```

Use cases: shortest unweighted route, social-network levels, minimum moves in grids, level-order exploration.

6. DFS — Depth-First Search

Idea: Go as deep as possible before backtracking. Implement using recursion or an explicit stack.

```
static void dfs(List<List<Integer>> g, int u, boolean[] vis) {
    vis[u] = true;
    System.out.print(u + " ");
    for (int v : g.get(u)) {
        if (!vis[v]) dfs(g, v, vis);
    }
}

static void dfsAll(List<List<Integer>> g) {
    boolean[] vis = new boolean[g.size()];
    for (int i = 0; i < g.size(); i++)
        if (!vis[i]) dfs(g, i, vis);
}
```

Complexity: O(V+E) time, O(V) visited + recursion stack. Deep recursion can overflow the Java call stack; use iterative DFS for very deep inputs.

```
static void iterativeDfs(List<List<Integer>> g, int src) {
    boolean[] vis = new boolean[g.size()];
    Deque<Integer> st = new ArrayDeque<>();
    st.push(src);
    while (!st.isEmpty()) {
        int u = st.pop();
        if (vis[u]) continue;
        vis[u] = true;
        for (int v : g.get(u))
            if (!vis[v]) st.push(v);
    }
}
```

7. Connected Components

In an undirected graph, each DFS/BFS started from an unvisited vertex discovers one connected component.

```
static int countComponents(List<List<Integer>> g) {
    int n = g.size(), count = 0;
    boolean[] vis = new boolean[n];
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            count++;
            dfs(g, i, vis);
        }
    }
    return count;
}
```

Complexity O(V+E). Variants: component sizes, largest component, number of provinces (matrix input), number of islands (grid input).

8. Cycle Detection

Undirected graph — DFS with parent

```
static boolean hasCycleUndirected(
        List<List<Integer>> g, int u, int parent, boolean[] vis) {
    vis[u] = true;
    for (int v : g.get(u)) {
        if (!vis[v]) {
            if (hasCycleUndirected(g, v, u, vis)) return true;
        } else if (v != parent) {
            return true;
        }
    }
    return false;
}
```

Run from every unvisited vertex to cover disconnected graphs. This parent method assumes a simple undirected graph; parallel edges require edge-ID handling.

Directed graph — DFS recursion-state method

```
// state: 0 = unvisited, 1 = in current recursion path, 2 = done
static boolean cycleDfs(List<List<Integer>> g, int u, int[] state) {
    state[u] = 1;
    for (int v : g.get(u)) {
        if (state[v] == 1) return true;
        if (state[v] == 0 && cycleDfs(g, v, state)) return true;
    }
    state[u] = 2;
```

```
    return false;
}
```

A back edge to state 1 proves a directed cycle. Complexity O(V+E). Alternative: Kahn's algorithm — if processed vertices < V, a directed cycle exists.

9. Bipartite Graph Check / Two-Colouring

Colour adjacent vertices with opposite colours. If an edge joins equal colours, the graph is not bipartite. BFS/DFS must cover every component.

```
static boolean isBipartite(List<List<Integer>> g) {
    int n = g.size();
    int[] color = new int[n];
    Arrays.fill(color, -1);
    Queue<Integer> q = new ArrayDeque<>();

    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        color[s] = 0; q.offer(s);
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : g.get(u)) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.offer(v);
                } else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}
```

Complexity O(V+E). A self-loop immediately makes a graph non-bipartite. Odd-cycle detection is a useful equivalent concept.

10. Topological Sorting — DAG Only

A topological order is a linear ordering where every directed edge u → v places u before v. It exists iff the directed graph is acyclic.

Kahn's algorithm (BFS / indegree)

```
static List<Integer> topoSort(List<List<Integer>> g) {
    int n = g.size();
    int[] indegree = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : g.get(u)) indegree[v]++;

    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.offer(i);

    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : g.get(u))
            if (--indegree[v] == 0) q.offer(v);
    }
    if (order.size() != n) return new ArrayList<>(); // cycle
    return order;
}
```

Complexity O(V+E). An empty result here indicates a cycle (assuming n > 0).

DFS topological sort

```
static void topoDfs(List<List<Integer>> g, int u,
                    boolean[] vis, Deque<Integer> order) {
    vis[u] = true;
    for (int v : g.get(u))
        if (!vis[v]) topoDfs(g, v, vis, order);
    order.push(u);
}
```

For DFS version, first detect cycles (use recursion states); reverse finishing order. Applications: course schedule, build dependencies, task pipelines.

11. Shortest Path Algorithms — Choose Correctly

Algorithm	Graph / weights	Typical time	Key note
BFS	Unweighted / equal weights	O(V+E)	Minimum edge-count path

Algorithm	Graph / weights	Typical time	Key note
Dijkstra	Non-negative weights	O((V+E) log V)	Greedy + min priority queue
Bellman–Ford	Negative edges allowed	O(VE)	Detects reachable negative cycle
Floyd–Warshall	All pairs; negative edges, no negative cycle for finite shortest paths	O(V³)	DP over intermediate vertices
DAG shortest path	Weighted DAG; negative weights allowed	O(V+E)	Topological order + relax edges

Bellman–Ford

```
static long[] bellmanFord(int n, int[][] edges, int src) {
    long INF = Long.MAX_VALUE / 4;
    long[] d = new long[n];
    Arrays.fill(d, INF); d[src] = 0;

    for (int i = 1; i < n; i++) {
        boolean changed = false;
        for (int[] e : edges) {
            int u=e[0], v=e[1], w=e[2];
            if (d[u] != INF && d[u] + w < d[v]) {
                d[v] = d[u] + w; changed = true;
            }
        }
        if (!changed) break;
    }
    for (int[] e : edges) {
        int u=e[0], v=e[1], w=e[2];
        if (d[u] != INF && d[u] + w < d[v])
            throw new IllegalStateException("Reachable negative cycle");
    }
    return d;
}
```

The code uses directed edges; add both directions for an undirected graph. It detects negative cycles reachable from src. Use long to reduce overflow risk.

Floyd–Warshall

```
// dist[i][j] = direct edge weight or INF; dist[i][i] = 0
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][k] < INF && dist[k][j] < INF)
                dist[i][j] = Math.min(dist[i][j],
                                      dist[i][k] + dist[k][j]);
// Negative cycle exists if any dist[i][i] < 0.
```

Time $O(V^3)$, space $O(V^2)$. The k loop must be outermost for the standard DP formulation.

12. Minimum Spanning Tree (MST)

A spanning tree connects all vertices of a connected, undirected graph with no cycles. A minimum spanning tree minimizes total edge weight and contains V−1 edges.

Kruskal — edge sorting + DSU

```
import java.util.*;

class DSU {
    int[] parent, size;
    DSU(int n) {
        parent = new int[n]; size = new int[n];
        for (int i=0;i<n;i++) { parent[i]=i; size[i]=1; }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        a=find(a); b=find(b);
        if (a==b) return false;
        if (size[a] < size[b]) { int t=a; a=b; b=t; }
        parent[b]=a; size[a]+=size[b]; return true;
    }
}
static long kruskal(int n, int[][] edges) {
    Arrays.sort(edges, Comparator.comparingInt(e -> e[2]));
    DSU dsu = new DSU(n);
    long total=0; int used=0;
    for (int[] e : edges) {
        if (dsu.union(e[0], e[1])) {
            total += e[2];
            if (++used == n-1) break;
        }
    }
```

```
    if (used != n-1) throw new IllegalArgumentException("Disconnected graph");
    return total;
}
```

Time O(E log E) plus almost-constant amortized DSU operations; DSU space O(V).

Prim — min priority queue

```
// Weighted adjacency list: List<List<Edge>>, Edge has to and weight.
static long prim(List<List<Edge>> g) {
    int n=g.size(), count=0;
    boolean[] used=new boolean[n];
    PriorityQueue<long[]> pq =
        new PriorityQueue<>(Comparator.comparingLong(a -> a[1]));
    pq.offer(new long[]{0,0});
    long total=0;
    while (!pq.isEmpty()) {
        long[] x=pq.poll();
        int u=(int)x[0]; long w=x[1];
        if (used[u]) continue;
        used[u]=true; count++; total+=w;
        for (Edge e : g.get(u))
            if (!used[e.to]) pq.offer(new long[]{e.to,e.weight});
    }
    if (count != n) throw new IllegalArgumentException("Disconnected graph");
    return total;
}
```

Time O((V+E) log V) with adjacency list + binary heap. Prim and Kruskal produce MSTs; neither is a single-source shortest-path algorithm.

13. Disjoint Set Union (Union-Find)

Supports connectivity queries and component merging. Essential in Kruskal, cycle detection in undirected edge streams, and offline connectivity.

Optimisations: Path compression in find + union by size/rank. Amortized complexity is O(α(V)) per operation, where α is the inverse Ackermann function (effectively constant for practical input sizes). See the DSU class in the Kruskal section.

14. Strongly Connected Components (SCC)

In a directed graph, an SCC is a maximal set where every vertex can reach every other vertex.

Kosaraju's algorithm

• DFS on original graph; push vertices by finishing time.

• Reverse every edge to form the transpose graph.

• Pop vertices in decreasing finishing order; DFS on transpose. Each DFS tree is one SCC.

Complexity O(V+E) time and O(V+E) space. Use iterative DFS if recursion depth may be large.

Tarjan's algorithm

One DFS computes discovery time, low-link values, and an active stack. When low[u] == disc[u], u is the root of an SCC; pop through u. Complexity O(V+E), one graph traversal. Know the low-link update distinction: tree edge uses low[child]; back edge to a vertex still on stack uses disc[neighbor].

15. Bridges and Articulation Points

Bridge: An undirected edge whose removal increases the number of connected components. **Articulation point:** A vertex whose removal (and incident edges) increases the number of components.

DFS low-link idea: disc[u] is discovery time; low[u] is the earliest discovery time reachable from u's DFS subtree using tree edges plus at most one back edge.

• Tree edge (u,v): after DFS(v), update low[u] = min(low[u], low[v]).

• Back edge (u,v) to an ancestor: update low[u] = min(low[u], disc[v]).

• Bridge condition: low[v] > disc[u].

• Articulation non-root u: some child v has low[v] >= disc[u].

• DFS root is articulation point iff it has at least two DFS-tree children.

Both algorithms run in O(V+E). Handle disconnected graphs and distinguish the parent edge by edge ID when parallel edges are possible.

16. Eulerian Path and Circuit

Euler trail/path: uses every edge exactly once. **Euler circuit:** uses every edge exactly once and returns to its starting vertex.

• Undirected Euler circuit: all non-isolated vertices are connected and every vertex has even degree.

• Undirected Euler trail (not circuit): connected non-isolated part and exactly two odd-degree vertices.

• Directed Euler circuit: all nonzero-degree vertices belong to one connected component in the appropriate underlying connectivity sense, and indegree = outdegree for every vertex.

• Directed Euler trail: one vertex has outdegree = indegree + 1 (start), one has indegree = outdegree + 1 (end), all others balance; connectivity condition also applies.

Hierholzer's algorithm — core Java pattern

```
static List<Integer> eulerCircuit(List<Deque<Integer>> g, int start) {
    Deque<Integer> stack = new ArrayDeque<>();
    List<Integer> path = new ArrayList<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int u = stack.peek();
        if (!g.get(u).isEmpty()) {
            int v = g.get(u).pollFirst();
            stack.push(v);
        } else {
            path.add(stack.pop());
        }
    }
    Collections.reverse(path);
    return path;
}
```

This is the traversal core for an adjacency structure where each undirected edge is represented consistently and removed from both endpoints (usually with edge IDs). Verify that path length equals E+1 and validate connectivity/degrees before accepting the result. Complexity O(V+E).

17. Hamiltonian Path and Circuit

Hamiltonian path: visits every vertex exactly once. **Hamiltonian cycle:** visits every vertex once and returns to start.

Unlike Eulerian problems, Hamiltonian problems do not have a simple degree-parity test. General Hamiltonian cycle is NP-complete; backtracking is exponential in the worst case.

Backtracking outline

```
boolean search(int u, int count) {
    if (count == n) return edgeExists(u, start); // cycle
    for (int v : neighbors(u)) {
        if (!visited[v]) {
            visited[v] = true;
            if (search(v, count + 1)) return true;
            visited[v] = false;
        }
    }
    return false;
}
```

Worst-case time O(V!) for a basic search. Bitmask DP (Held–Karp style) for Hamiltonian path/cycle is $O(V^2 \cdot 2^V)$ time and $O(V \cdot 2^V)$ space. This is a hard-problem topic, not a standard greedy solution.

18. Graph Problems on Grids

• Number of Islands: treat each land cell as a vertex; 4-direction or 8-direction neighbours as specified; DFS/BFS O(RC).

• Flood Fill: BFS/DFS from a starting cell; recolour all connected cells with the target colour.

• Rotting Oranges: multi-source BFS; push all initially rotten cells at time 0.

• Shortest Path in Binary Matrix: BFS when each move has equal cost.

• Walls and Gates / distance to nearest source: multi-source BFS.

• Word Ladder: BFS on implicit graph; each word is a vertex and one-character changes are edges.

• 01 Matrix: multi-source BFS from all zero cells.

Grid checklist: bounds check, visited marking, direction arrays, obstacles, diagonal movement rule, and whether multiple starting points exist.

19. Other Important Graph Patterns

Multi-source BFS

```
int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
Queue<int[]> q = new ArrayDeque<>();
// Add every source cell to q and mark its distance as 0.
// Then perform ordinary BFS; first discovery gives minimum steps.
```

0–1 BFS

For edge weights only 0 or 1, use a deque: push a 0-weight relaxation to the front and a 1-weight relaxation to the back. O(V+E). It is not ordinary BFS and not general Dijkstra.

DAG shortest / longest path

Topologically order the DAG, then relax outgoing edges in that order. Works with negative weights because a DAG has no directed cycle. O(V+E).

Minimum Spanning Forest

For a disconnected undirected weighted graph, run Kruskal across all components or start Prim from every unvisited vertex. Result is a minimum spanning forest, not one spanning tree.

Greedy graph colouring heuristic

Choose a vertex order and assign each vertex the smallest colour not used by its already-coloured neighbours. This is a heuristic; result depends on ordering and is not guaranteed to use the minimum number of colours. Basic implementation O(V+E) with suitable neighbour-colour marking.

Maximum flow / minimum cut

Know the residual graph, augmenting path, bottleneck capacity, and max-flow min-cut theorem. Edmonds–Karp uses BFS augmenting paths and runs $O(VE^2)$. Dinic is commonly used for better practical performance; complexity depends on graph class (general bound $O(V^2E)$). This is not a greedy MST/shortest-path algorithm.

Biconnected components

Vertex-biconnected components are maximal subgraphs that remain connected after removing any one vertex (with conventional edge-stack definitions). Tarjan low-link DFS is the standard approach; often paired with articulation points.

20. Graph Traversal and Algorithm Complexity Sheet

Topic	Time	Extra space / note
Adjacency matrix storage	$O(V^2)$	$O(V^2)$
Adjacency list storage	O(V+E)	O(V+E)
BFS / DFS	O(V+E)	O(V)
Connected components	O(V+E)	O(V)
Cycle detection	O(V+E)	O(V)
Bipartite check	O(V+E)	O(V)
Topological sort	O(V+E)	O(V)
Dijkstra (heap)	O((V+E) log V)	O(V+E)
Bellman–Ford	O(VE)	O(V)
Floyd–Warshall	$O(V^3)$	$O(V^2)$
Kruskal	O(E log E)	O(V) DSU
Prim (heap)	O((V+E) log V)	O(V+E)

Topic	Time	Extra space / note
SCC / bridges / articulation	O(V+E)	O(V) auxiliary
Eulerian traversal	O(V+E)	O(V+E)

21. High-Tech Interview: How to Recognise the Pattern

• Minimum number of steps in an unweighted graph → BFS.

• Explore all reachable nodes / count components / flood fill → DFS or BFS.

• Prerequisites / course ordering / dependency resolution → Topological sort; cycle means impossible.

• Can nodes be split into two groups with no same-group edge? → Bipartite BFS/DFS.

• Cheapest way to connect all locations → MST (Kruskal or Prim).

• Cheapest route from one source, weights non-negative → Dijkstra.

• Negative edge weights → Bellman–Ford; all-pairs distances → Floyd–Warshall when V is small.

• Dynamic connectivity / cycle check while adding undirected edges → DSU.

• Critical connection / single point of failure → Bridges / articulation points.

• Mutual reachability groups in directed graph → SCC.

• Use every edge exactly once → Eulerian trail/circuit; visit every vertex exactly once → Hamiltonian path/cycle.

• Grid shortest distance with equal moves → BFS; multiple sources → multi-source BFS; weights 0/1 → 0–1 BFS.

22. Common Interview Mistakes

• Using Dijkstra with negative edges.

• Using BFS for arbitrary weighted edges.

• Confusing MST with shortest-path tree: MST minimizes total tree edge weight, not each source-to-vertex route.

• Running DFS/BFS from only vertex 0 and missing disconnected components.

• Marking visited too late in BFS.

• Using a directed-cycle parent check intended for undirected graphs, or vice versa.

• Assuming every graph has a topological order (only DAGs do).

• Assuming greedy coin selection is always optimal.

• Not handling stale entries in PriorityQueue-based Dijkstra.

• Using int for large path sums; prefer long when constraints require it.

• Not checking disconnected input before claiming an MST.

• Confusing Eulerian (edges) with Hamiltonian (vertices).

23. Practice Ladder — Graphs Only

Foundation: Represent an undirected graph; print adjacency list; BFS; recursive and iterative DFS.

Traversal: Count components; component sizes; path exists; shortest path in an unweighted graph.

Properties: Detect cycle in undirected/directed graph; check bipartite; find eventual safe states.

DAG: Topological sort using Kahn and DFS; Course Schedule I/II; DAG shortest path.

Shortest path: Network Delay Time; Path With Minimum Effort; Cheapest Flights Within K Stops; Bellman–Ford; Floyd–Warshall.

MST / DSU: Redundant Connection; Number of Provinces; Kruskal; Prim; Min Cost to Connect All Points.

Advanced DFS: Bridges; articulation points; SCC using Kosaraju and Tarjan; critical connections.

Grid / BFS: Number of Islands; Rotting Oranges; 01 Matrix; Word Ladder; Shortest Path in Binary Matrix.

Hard / specialised: Reconstruct Itinerary (Euler trail); Hamiltonian path for small n; max flow; graph colouring heuristic.

24. Recommended Study Order

• Graph terminology + representations

• BFS and DFS (including disconnected graphs)

• Components, cycle detection, bipartite

• Topological sorting and DAG problems

• Unweighted BFS shortest path, then Dijkstra

• Bellman–Ford, Floyd–Warshall, 0–1 BFS

• DSU, Kruskal, Prim

• SCC, bridges, articulation points

• Eulerian algorithms, Hamiltonian backtracking/bitmask DP

• Grid patterns, flow, advanced interview problems

Final rule: For every graph problem, first identify: directed or undirected, weighted or unweighted, connected or disconnected, constraints on V and E, and whether the task asks about edges, vertices, reachability, shortest path, or total connection cost.